PRD: PostDaily (working title)

AI-powered daily social media marketing platform for small businesses. Frontend release, built phase by phase in Antigravity IDE.

Version 1.0 | Scope: frontend only, backend-ready

1. Overview

1.1 Product summary

PostDaily helps a small-business owner publish one good social media post every day. Each day the owner generates a single post (image, caption, hashtags, call to action), reviews and edits it, and keeps it in a permanent history. The product never invents numbers: if a metric was not recorded, it says so.

1.2 Problem

Small-business owners lack time and design or copywriting skills to post consistently. Existing tools are either too complex, or generate content in bulk that feels generic and is never reviewed.

1.3 Target users

- Primary: owner-operators of small businesses (cafes, salons, local retail, studios, freelancers) who manage their own social media.
- Secondary: a small team member or freelancer managing several businesses (hence the business switcher).

1.4 Goals

1. A complete, working, responsive frontend with all six sections: Dashboard, Create Today's Post, All Posts, Analytics, Business Profile, Settings.
2. All Posts is a reliable historical record: after 25, 100 or 500 days, every post stays reachable with image, caption, date, likes, shares and comments.
3. Honest data everywhere: no fabricated statistics, no fake publishing, no fake metric fetching.
4. Backend-ready: swapping demo data for a Node.js/Express API (DeepSeek for text, Hugging Face for images, Supabase for storage) requires changes in the services layer only.
5. Builds with zero errors and zero console errors.

1.5 Non-goals for this release

- Real authentication, real publishing to social networks, real metric syncing, billing.
- Bulk generation of 30 posts in advance. The app generates today's post only.
- Any API keys or secrets in frontend code.

2. Product principles

1. **Honest data.** Missing metrics show 'Not recorded' or a dash. Zero is only shown when zero was recorded. No placeholder numbers in totals, charts or cards.
2. **One post a day.** The workflow is built around today's post, not content calendars.
3. **History is permanent.** All Posts is the source of truth and never truncates to 'latest few'.
4. **Demo data is labelled and isolated.** It lives in its own folder, is flagged on every record, and is loaded only in demo mode with a visible banner.
5. **Do not pretend.** Publishing and metric sync are shown as unavailable until a real integration confirms them.
6. **Backend-ready by design.** UI talks to service interfaces, never to mock files directly.
7. **Calm, professional UI.** Restrained motion, readable type, no gimmicks.

3. Tech stack and architecture

3.1 Stack

Area	Choice
Framework	React + Vite
Language	TypeScript (strict)
Styling	Tailwind CSS with design tokens in the Tailwind config
Icons	lucide-react (no emojis as icons)
Routing	React Router (v6+)
Charts	Recharts (recommended addition)
Utilities	clsx, date-fns (recommended additions)
Data hooks	Custom hooks over services, with loading and error state (TanStack Query optional)

3.2 Folder structure

```
src/
  app/            router, providers, app shell
  components/
    ui/           Button, Badge, Input, Select, Modal, Tabs, Menu, Toast,
Skeleton, EmptyState
    layout/       Sidebar, MobileNav, BusinessSwitcher, UserMenu, DemoBanner
    posts/        PostCard, PostListItem, PostGrid, PostDetailModal,
PostFilters, MetricStat
    charts/       EngagementChart, PlatformComparison, ChartCard
  pages/          DashboardPage, CreatePostPage, AllPostsPage, AnalyticsPage,
ProfilePage, SettingsPage
  services/
    types.ts      service interfaces
    mock/         mock implementations (read from data/demo)
    http/         http implementations (fetch to API_BASE_URL)
    index.ts      picks implementation from VITE_DATA_SOURCE
  data/demo/      demo businesses and posts (clearly marked, isolated)
  hooks/          usePosts, usePost, useBusiness, useGeneratePost,
useAnalytics
  types/          domain types
  lib/            format, clipboard, download, dates, query-params
```

3.3 Data access rule

Pages and components only call hooks. Hooks only call service interfaces (`PostsService` , `GenerationService` , `AnalyticsService` , `BusinessService`). Two implementations exist: `mock` and `http` . The active one is chosen by `VITE_DATA_SOURCE` ('mock' | 'api'). Replacing mock with real API calls must not touch any page or component.

3.4 Environment variables (frontend)

Only non-secret values: `VITE_DATA_SOURCE` , `VITE_API_BASE_URL` . No API keys of any kind. DeepSeek, Hugging Face and Supabase credentials live only in the future Express backend.

4. Design system

4.1 Tokens

Token	Value (starting point)
Background	#FAF8F4 warm off-white
Surface	#FFFFFF
Border	#E8E4DC
Text	#1F2421 charcoal
Muted text	#6B6F6A
Accent (green)	#2F7D5B, hover #266849, soft #E8F3EC
Danger / Warning / Info	muted red, amber, blue, used only for status

4.2 Style rules

- Clean sans-serif (Inter or Geist), clear type scale, generous spacing.
- Radius 8 to 12px maximum. No pill-everything, no oversized rounded cards.
- Subtle 1px borders, soft low-opacity shadows.
- Motion: 150 to 200ms transitions for hover, menus and modals only. Respect `prefers-reduced-motion`.
- Avoid: purple gradients, emojis as icons, decorative animation, generic AI landing-page look, fake stats.
- Status badges: Draft (neutral), Ready to Publish (blue), Published (green), Failed (red).
- Focus rings visible on all interactive elements. Contrast at least WCAG AA.

5. Data model

```
type Platform = 'instagram' | 'facebook' | 'linkedin' | 'x' | 'tiktok';
type PostStatus = 'draft' | 'ready' | 'published' | 'failed';

// null = not recorded. 0 = recorded as zero.
interface PostMetrics {
  likes: number | null;
  shares: number | null;
  comments: number | null;
  reach: number | null;
```

```
  clicks: number | null;
  recordedAt: string | null;
}

interface Post {
  id: string;
  businessId: string;
  date: string;              // YYYY-MM-DD, one daily post per business per
date
  platform: Platform;
  topic: string;
  objective: string;
  tone: string;
  caption: string;
  hashtags: string[];
  cta: string;
  imageUrl: string | null;
  visualPrompt: string;
  status: PostStatus;
  publishedAt: string | null;
  publishUrl: string | null;
  failureReason: string | null;
  metrics: PostMetrics | null;
  isDemo: boolean;           // true for every demo record
  createdAt: string;
  updatedAt: string;
}

interface BusinessProfile {
  id: string; name: string; industry: string; description: string;
  location: string; targetAudience: string; marketingGoals: string[];
  platforms: Platform[]; brandTone: string; contentPreferences: string;
}

interface ListPostsParams {
  search?: string;
  platforms?: Platform[];
  statuses?: PostStatus[];
  dateFrom?: string; dateTo?: string;
  sort: 'newest' | 'oldest' | 'engagement';
  page: number; pageSize: number;
}
interface ListPostsResult { items: Post[]; total: number; hasMore: boolean; }
```

Engagement = likes + shares + comments, counting only recorded values. Posts with no recorded metrics sort last under 'engagement'.

6. Functional requirements by page

6.1 App shell

- Fixed left sidebar on desktop with logo, business switcher, nav (Dashboard, Create Today's Post, All Posts, Analytics, Business Profile, Settings) and a user profile menu at the bottom.
- Mobile: top bar plus slide-over drawer or bottom navigation. Keyboard and screen-reader accessible.
- Demo banner when `VITE_DATA_SOURCE=mock`: 'Demo mode: sample data, nothing is published.'
- Business switcher changes the active business for all pages.

6.2 All Posts (highest priority)

Header: title 'All Posts' and total post count for the active business (count reflects filters: 'Showing 12 of 28').

Controls:

- Search by caption or topic (debounced).
- Filters: date range, platform (multi-select), status (multi-select), with a clear-all action.
- Sort: Newest, Oldest, Engagement.
- View toggle: grid (default) and list. Choice persisted.
- Filters, sort and search are stored in URL query params so views are shareable and survive refresh.

Post card (every card):

1. Generated image (with graceful placeholder if missing).
2. Caption clamped to a few lines with 'Show more' / 'Show less'.
3. Platform name and post date.
4. Day label (for example 'Day 12' or weekday and date).
5. Likes, shares, comments with lucide icons and readable numbers.
6. Reach and clicks when available.
7. Status badge.
8. Actions: View Details, Edit, Regenerate, Copy Caption, Download Image (primary action visible, others in a menu on small screens).
9. 'Demo' badge when `isDemo`.

Unrecorded metrics show 'Not recorded' or a dash, never an invented value.

Detail view: clicking a card opens a modal (route `/posts/:id`, deep-linkable) with full image, full caption, hashtags, CTA, platform, status, publication details (publish date and link, or 'Not published'), failure reason if failed, and a metrics panel with likes, shares, comments, reach, clicks and 'last recorded' time. Includes Edit, Regenerate, Copy, Download.

Scaling: server-style pagination via 'Load more' (page size 12) with 'Showing X of Y'. The full history stays accessible; nothing is capped. Layout stays fast with 500+ posts (lazy-loaded images, stable card heights).

States: skeleton loading, error with retry, empty history ('No posts yet' with a button to Create Today's Post), empty search ('No posts match your filters' with Clear filters).

6.3 Create Today's Post

- Business context panel (name, industry, audience, tone, goals) with link to edit profile.
- Controls: platform selection, objective selection (for example Promote offer, Build awareness, Drive visits, Engage community, Announce news), tone selection.
- 'Generate Post' button. Only today's post is generated. If today's post already exists, show it and offer Regenerate instead of silently creating a duplicate.
- Result workspace: image preview, editable caption, hashtags, CTA, and visual prompt fields.
- Actions: regenerate caption, regenerate image (independent of each other), copy caption, download image, save as draft, mark ready.
- Platform-style preview that updates live as fields are edited (Instagram, Facebook, LinkedIn, X layouts).
- Loading states for caption and image separately. Errors show a clear message and a Retry button. Failed image does not discard a good caption.
- 'Publish' is disabled with an explanation ('Connect a social account to publish') until a real integration exists. Never show 'Published' unless the API says so.
- In mock mode the generator returns clearly labelled sample content with local placeholder images, and the UI notes it is sample output.

6.4 Dashboard

- Personalized welcome (user and business name).
- Today's generation status: not generated, draft, ready, published, failed.
- Prominent 'Generate Today's Post' button (or 'Review Today's Post' when it exists).
- Preview of today's post when present.
- Recent posts (latest 4 to 6) with link to All Posts.

- Compact totals: posts, likes, shares, comments, computed only from recorded data. Show a dash when nothing is recorded.
- Small engagement trend chart only when enough metrics exist (at least 3 posts with recorded metrics); otherwise an explanatory empty state.

6.5 Analytics

- Date-range filter (7 days, 30 days, 90 days, custom).
- Engagement trend over time; likes, shares, comments, reach and clicks as selectable series or tabs.
- Top-performing posts list (links to detail view).
- Platform comparison.
- Summary figures computed from recorded data only, with a note on how many posts had recorded metrics.
- Empty state when data is insufficient. No fabricated analytics.

6.6 Business Profile

Editable form: business name, industry, description, location, target audience, marketing goals, preferred platforms, brand tone, content preferences. Validation, unsaved-changes warning, save confirmation. In mock mode, edits persist locally for the session or in local storage, and are labelled as local.

6.7 Settings

Tabs or sections: Account (name, email, display only plus edit form), Appearance (light, dark or system theme, density), Notifications (daily reminder toggle and time, email preferences). Settings saved locally in mock mode.

7. Phased build plan for Antigravity IDE

Build one phase per Antigravity task or conversation. Each phase ends with a working app, a clean build and a short review before moving on. Do not start the next phase until the acceptance criteria pass.

Phase 0: Foundation

Goal: a clean, running project skeleton.

Scope: Vite React TypeScript project, Tailwind with design tokens, lucide-react, React Router, ESLint and Prettier, strict TS, folder structure from section 3.2, `.env.example`, README with run instructions, `docs/PRD.md` containing this document.

Acceptance: `npm run dev` and `npm run build` pass; placeholder routes render for all six pages; no console errors.

Antigravity prompt:

> Read docs/PRD.md sections 1 to 4. Scaffold the project with Vite, React, TypeScript, Tailwind, lucide-react and React Router using the folder structure in section 3.2. Configure Tailwind with the design tokens from section 4. Add routes for all six pages with placeholder content. Add .env.example (VITE_DATA_SOURCE, VITE_API_BASE_URL only). Run the build and fix all errors. Do not add features beyond this scope.

Phase 1: Design system and app shell

Goal: the layout feels like a real product.

Scope: reusable UI components (Button, Badge, Input, Select, Checkbox, Modal, Tabs, Dropdown menu, Toast, Skeleton, EmptyState), Sidebar with logo, business switcher and user menu, responsive mobile navigation, demo banner, page header component.

Acceptance: sidebar fixed on desktop; mobile nav works at 375px; keyboard navigation and focus rings work; modal traps focus and closes on Escape; components documented by a simple internal showcase route (optional, removed or hidden before release).

Antigravity prompt:

> Read PRD sections 4 and 6.1. Build the design system components and the app shell. Follow the tokens exactly: warm off-white background, white surfaces, charcoal text, green accent, modest radius, soft shadows. Make everything accessible and responsive. Verify in the browser at 375px, 768px and 1440px and fix layout issues.

Phase 2: Types, services and demo data

Goal: a backend-ready data layer.

Scope: domain types from section 5; service interfaces; mock implementations with simulated latency and error simulation; `src/data/demo` with 2 demo businesses and about 28 demo posts across at least 28 consecutive days, mixed platforms and statuses, some with unrecorded metrics (null), one failed post with a reason; every record `isDemo: true`; local SVG placeholder images in `public/demo`; hooks; service selector via `VITE_DATA_SOURCE`; empty `http` implementation stubs with the endpoint contract from section 9.

Acceptance: no component imports from `data/demo`; list, filter, sort and pagination logic works in the mock service; unit tests (or a test script) cover engagement sorting and null handling.

Antigravity prompt:

> Read PRD sections 3 and 5. Create the types, service interfaces, mock services and demo data. Demo data must be clearly flagged isDemo and kept in src/data/demo. Include 28 posts so pagination can be tested, with some metrics set to null. Implement listPosts with search, platform and status filters, date range, sorting (newest, oldest, engagement) and page/pageSize. Only hooks may call services. Add tests for sorting and null handling.

Phase 3: All Posts page

Goal: the complete, scalable post history (top priority).

Scope: everything in section 6.2 except the detail modal: header with count, search, filters, sort, URL-synced state, grid and list views, PostCard and PostListItem with all required fields, MetricStat component, Show more, status badges, action menu, Load more with 'Showing X of Y', skeleton, error and empty states.

Acceptance: all 28 demo posts reachable through Load more; filters and sort combine correctly; refresh keeps filters; unrecorded metrics show 'Not recorded' or a dash; card actions work (Copy Caption copies, Download Image downloads); no layout shift or console errors.

Antigravity prompt:

> Read PRD section 6.2 carefully. Build the All Posts page with every required control and card field. State for search, filters, sort and page must live in URL query params. Use the usePosts hook only. Never show invented metric values. Test with the 28 demo posts: load all of them via Load more, apply combined filters, switch views, and confirm results. Report anything not working.

Phase 4: Post detail view and edit

Goal: open any post and see everything.

Scope: PostDetailModal on route `/posts/:id` (deep link works on refresh), full caption, hashtags, CTA, image, publication details, metrics panel, edit dialog or edit mode (caption, hashtags, CTA, status draft or ready), Regenerate entry point, copy and download actions, next and previous post navigation within the current list.

Acceptance: opening, closing and deep linking work; edits update the list view without reload; failed posts show the failure reason; unpublished posts show 'Not published'.

Antigravity prompt:

> Read PRD section 6.2 (Detail view). Build the post detail modal bound to /posts/:id and the edit flow. Keep the list state when the modal closes. Make the modal accessible (focus trap, Escape, labelled title). Test deep links and edits in the browser.

Phase 5: Create Today's Post

Goal: the generation workspace.

Scope: section 6.3 in full: context panel, platform, objective and tone selectors, Generate Post, separate caption and image loading states, error and Retry, editable fields, visual prompt, regenerate caption and image independently, copy, download, save draft, mark ready, platform-style previews, today's-post-exists guard, disabled Publish with explanation.

Acceptance: generation uses `GenerationService` only; simulated failure shows retry and keeps good content; saved post appears at the top of All Posts; second generation on the same day does not create a duplicate.

Antigravity prompt:

> Read PRD section 6.3. Build the Create Today's Post page against the GenerationService interface (mock only for now). Simulate latency and occasional failures to verify loading, error and retry states. Add live platform previews for Instagram, Facebook, LinkedIn and X. Do not add any real API calls or keys. Save results through PostsService and confirm they appear in All Posts.

Phase 6: Dashboard

Goal: a useful daily home screen.

Scope: section 6.5 above is Analytics; this phase is section 6.4: welcome, today's status, generate or review button, today's preview, recent posts, totals from recorded data, trend chart with sufficient-data guard, empty states.

Acceptance: with empty data every widget shows a helpful empty state and no numbers; with demo data totals match the recorded values only; the chart is hidden when fewer than 3 posts have metrics.

Antigravity prompt:

> Read PRD section 6.4. Build the Dashboard using existing hooks and components. Compute totals only from recorded metrics (ignore nulls, never treat null as zero). Test both the empty state (temporarily switch to an empty mock dataset) and the demo state.

Phase 7: Analytics

Goal: honest performance insight.

Scope: section 6.5: date-range filter, engagement trend, metric tabs (likes, shares, comments, reach, clicks), top-performing posts, platform comparison, summaries with 'based on N posts with recorded metrics', empty state.

Acceptance: charts respond to the date range; posts without metrics are excluded and counted in a note; every chart has an accessible text summary or table alternative.

Antigravity prompt:

> Read PRD section 6.5. Build the Analytics page with Recharts using AnalyticsService. Exclude null metrics, show how many posts contributed, and show an empty state when data is insufficient. Verify date range changes, then check at mobile width.

Phase 8: Business Profile and Settings

Goal: the context that drives generation.

Scope: sections 6.6 and 6.7: validated profile form, platform and goals multi-select, tone and content preferences, unsaved-changes guard, settings tabs (Account, Appearance with working theme, Notifications), local persistence in mock mode.

Acceptance: profile edits are reflected in Create Today's Post context panel and sidebar business switcher; theme switch works without flash; forms are keyboard accessible.

Antigravity prompt:

> Read PRD sections 6.6 and 6.7. Build Business Profile and Settings with validation and unsaved-changes handling. Wire profile changes into the Create Today's Post context panel. Implement a working light, dark and system theme using the design tokens.

Phase 9: Polish, accessibility and QA

Goal: release-quality frontend.

Scope: responsive pass at 375, 768, 1024, 1440; accessibility audit (labels, roles, contrast, focus order, reduced motion); performance (lazy routes, lazy images, memoized lists); remove unused code; consistent empty, loading and error states; console error sweep; README update.

Acceptance: section 10 checklist fully green; `npm run build` and lint pass with no warnings.

Antigravity prompt:

> Run a full QA pass against PRD section 10. Open every page at 375px, 768px and 1440px using the browser, fix layout and accessibility issues, remove console errors and unused code, and report a checklist of what passed and what was fixed. Do not add new features.

Phase 10: Backend handoff

Goal: make the later integration a swap, not a rewrite.

Scope: finalize `http` service implementations against section 9 contract, error mapping, auth header hook (token from future auth, no secrets), `docs/API_CONTRACT.md`, short guide for connecting Express, DeepSeek, Hugging Face and Supabase.

Acceptance: switching `VITE_DATA_SOURCE=api` points all services at `VITE_API_BASE_URL`; no page or component changes are needed.

Antigravity prompt:

> Read PRD section 9. Implement the http services and write docs/API_CONTRACT.md. Do not call any third-party AI or database directly from the frontend. Confirm that toggling VITE_DATA_SOURCE is the only change needed.

8. Working in Antigravity

- Save this PRD as `docs/PRD.md` in the project so every agent task can read it.
- Put the global rules below in the workspace rules so every task follows them.
- Run one phase per task. Review the agent's plan and artifacts before approving large changes.
- After each phase ask the agent to run the app in the browser and verify the acceptance criteria, then commit.
- Keep tasks narrow: if a phase grows, split it rather than letting the agent expand scope.

Global rules (paste into workspace rules):

1. Follow docs/PRD.md. Do not add features outside the current phase.
2. TypeScript strict. No `any` unless justified in a comment.
3. Pages and components never import demo data or call fetch directly. Use hooks, hooks use services.
4. Never display fabricated numbers. null means 'Not recorded'. Zero is shown only when recorded.
5. Demo data stays in src/data/demo and every record has isDemo true.
6. No API keys or secrets in frontend code, env files or comments.
7. Use lucide-react icons only. No emojis as icons.
8. Every interactive element is keyboard accessible with visible focus.
9. Run the build and fix all errors and console errors before finishing a task.

9. Backend contract (for future Express API)

All responses JSON. The frontend sends the active `businessId`. Secrets stay server-side.

Method and path	Purpose
GET /api/businesses	List businesses for the user
GET, PUT /api/businesses/:id	Read or update business profile
GET /api/businesses/:id/posts	List posts: search, platforms, statuses, dateFrom, dateTo, sort, page, pageSize; returns items, total, hasMore
GET /api/posts/:id	Post detail
PATCH /api/posts/:id	Edit caption, hashtags, CTA, status
POST /api/businesses/:id/posts/generate	Generate today's post (DeepSeek caption, Hugging Face image); returns draft post
POST /api/posts/:id/regenerate-caption	Regenerate caption only
POST /api/posts/:id/regenerate-image	Regenerate image only
GET /api/businesses/:id/analytics	Aggregated recorded metrics, query by date range

Backend responsibilities: Supabase storage for posts and images, enforcing one post per business per date, recording metrics only when a real source reports them, and returning `null` for anything unrecorded.

10. Quality bar and definition of done

Functional

- All six routes work, including deep links and browser back and forward.
- All Posts: search, three filters, three sorts, grid and list toggle, Show more, Load more, detail modal, and all five card actions work.
- All 28 demo posts are reachable; adding posts never hides older ones.
- Create Today's Post: loading, error and retry states verified; independent caption and image regeneration; previews update live.
- Dashboard and Analytics show empty states when data is missing.

Honesty

- No fabricated metrics, totals, charts or statuses anywhere.
- Demo data labelled in the UI and isolated in code.
- No fake publishing; Publish disabled until an integration exists.

Quality

- Responsive at 375, 768, 1024 and 1440px with no horizontal scroll.
- Accessible: labelled controls, focus management in modals and menus, AA contrast, reduced-motion support.
- `npm run build` and lint pass; zero console errors or warnings in normal use.
- No secrets in the repo.

11. Risks and open questions

- **Regeneration history:** should regenerating replace the post content or keep versions? Default for v1: replace, with versioning as a later enhancement.
- **Multiple platforms per day:** v1 treats one daily post per business with a chosen platform. Cross-posting variants may come later.
- **Metric sources:** which social APIs will report likes, shares, comments, reach and clicks, and how often, is undecided. The UI already handles partial and missing data.
- **Image storage:** generated images will need Supabase storage and signed URLs; the frontend should only handle URLs.
- **Authentication:** deferred. Plan the user and business model so Supabase Auth can be added without UI rework.
- **Brand name:** PostDaily is a working title.